

AUDITORIA E PROPOSTA DE REORGANIZAÇÃO DA NAVEGAÇÃO DO ERP ORBE

CONTEXTO

O ERP ORBE já está em estágio avançado de homologação e refinamento funcional.

Grande parte dos fluxos principais já foi implementada, testada e refinada na prática, incluindo ajustes de usabilidade, correções de opções que não funcionavam, regras operacionais, fluxos financeiros, RH, aprovações, lotes, jornadas, faturamento e integrações.

Durante o uso real do sistema foi identificada uma questão importante de UX / arquitetura da informação.

Atualmente, o ORBE aparenta ter sido modelado principalmente segundo uma lógica de **setorização interna**, por exemplo:

- Encarregado / Operações
- Admin / RH
- Financeiro
- Cadastros
- Configurações

Entretanto, os fluxos reais não percorrem necessariamente os mesmos setores ou as mesmas etapas.

Exemplos:

- Operação por Volume possui lançamento operacional, aprovação/validação, impacto em RH, faturamento e financeiro.
- CLT possui ponto, jornadas, inconsistências, horas, aprovação e/ou lotes específicos.
- Intermitentes possuem importação, jornadas, horas, aprovação, lotes e pagamentos.
- Diaristas possuem lançamento, aprovação, lotes e pagamentos.
- Serviços Extras seguem uma lógica própria de lançamento e faturamento.
- Custos Extras seguem uma lógica própria relacionada a despesas e pagamentos.

Isso pode gerar dificuldade para novos usuários, porque para concluir um único processo o usuário precisa compreender previamente **como o ERP foi dividido internamente entre setores**.

PROBLEMA DE USABILIDADE IDENTIFICADO

Hoje, o raciocínio do usuário tende a ser:

“Em qual setor do sistema está a próxima etapa?”

O objetivo é aproximar o ORBE de um modelo mental mais natural:

“O que estou tratando agora?”

Por exemplo:

Se o usuário está tratando **Diaristas**, ele deveria conseguir entrar no módulo “Diaristas” e encontrar ali todas as opções pertinentes àquele fluxo, respeitando naturalmente as permissões do perfil.

Exemplo conceitual:

Diaristas

- Visão Geral
- Novo Lançamento
- Lançamentos
- Aprovações
- Lotes
- Pagamentos
- Histórico

Da mesma forma:

Intermitentes

- Visão Geral
- Jornadas
- Importações
- Inconsistências
- Aprovação RH
- Lotes
- Horas
- Pagamentos
- Histórico

CLT

- Visão Geral
- Ponto
- Jornadas

- Inconsistências
- Aprovações
- Horas
- Lotes
- Histórico

Operações por Volume

- Visão Geral
- Nova Operação
- Operações
- Aprovações
- Pendências
- Faturamento
- Financeiro
- Histórico

Serviços Extras

- Visão Geral
- Novo Serviço
- Lançamentos
- Aprovações
- Faturamento
- Histórico

Custos Extras

- Visão Geral
- Novo Custo
- Lançamentos
- Aprovações
- Contas a Pagar / Pagamentos
- Histórico

OBJETIVO DA AUDITORIA

Realizar uma análise completa da arquitetura atual do frontend e da navegação do ORBE para verificar a viabilidade de migrar a experiência de:

NAVEGAÇÃO ORIENTADA A SETORES

para:

NAVEGAÇÃO ORIENTADA A FLUXOS / CATEGORIAS

IMPORTANTE:

Nesta etapa, NÃO realizar nenhuma alteração no código.

Não refatorar.

Não mover páginas.

Não alterar rotas.

Não alterar permissões.

Não alterar backend.

Não alterar banco de dados.

Não alterar Edge Functions.

Não alterar RPCs.

Não alterar triggers.

Não alterar motores financeiros.

Não alterar automações.

Não alterar regras de negócio.

Não alterar RLS.

O objetivo inicial é exclusivamente:

ANALISAR → MAPEAR → IDENTIFICAR RISCOS → PROPOR ARQUITETURA.

PRINCÍPIO CENTRAL

A reorganização pretendida deve ser prioritariamente uma mudança de:

- Arquitetura da Informação
- Navegação
- Sidebar
- Organização visual
- Contextualização das páginas

e NÃO uma reconstrução dos fluxos internos.

Devemos preservar ao máximo:

- componentes existentes;
- páginas existentes;
- motores existentes;
- services;
- hooks;
- regras de negócio;
- banco;
- integrações;
- permissões;
- status;
- cálculos;
- automações.

A solução ideal deve seguir o conceito:

MESMO MOTOR + MESMAS REGRAS + NOVA FORMA DE NAVEGAR.

HIPÓTESE A SER INVESTIGADA

Verificar se é possível reutilizar uma mesma funcionalidade em diferentes contextos.

Exemplo:

Hoje pode existir uma única página de Aprovações RH.

Em vez de duplicar essa página, poderíamos eventualmente utilizar a mesma infraestrutura em contextos diferentes.

Exemplo conceitual:

`/diaristas/aprovacoes`

abre a página/componente de aprovação existente com filtro:

`tipo = DIARISTA`

Enquanto:

`/intermitentes/aprovacoes`

utiliza o mesmo motor/componente com:

`tipo = INTERMITENTE`

E:

/clt/aprovacoes

utiliza:

tipo = CLT

Isso é apenas uma hipótese.

Verifique no código se essa abordagem é possível, segura e adequada.

ARQUITETURA DE REFERÊNCIA A SER AVALIADA

Considere como hipótese inicial uma sidebar semelhante a:

ORBE

- Início
- Operações por Volume
 - Visão Geral
 - Nova Operação
 - Operações
 - Aprovações
 - Faturamento
 - Histórico
- CLT
 - Visão Geral
 - Ponto
 - Jornadas
 - Inconsistências
 - Aprovações
 - Horas
 - Lotes
 - Histórico
- Intermitentes
 - Visão Geral
 - Jornadas
 - Importações
 - Inconsistências
 - Aprovações
 - Horas
 - Lotes
 - Pagamentos
 - Histórico

- Diaristas
 - Visão Geral
 - Novo Lançamento
 - Lançamentos
 - Aprovações
 - Lotes
 - Pagamentos
 - Histórico
- Serviços Extras
 - Visão Geral
 - Novo Serviço
 - Lançamentos
 - Aprovações
 - Faturamento
 - Histórico
- Custos Extras
 - Visão Geral
 - Novo Custo
 - Lançamentos
 - Aprovações
 - Pagamentos
 - Histórico
- Financeiro
 - Visão Geral
 - Contas a Receber
 - Contas a Pagar
 - Faturamento
 - Caixa
 - Bancos
 - Conciliação
 - DRE
 - Competências
- Relatórios
- Cadastros
- Configurações

IMPORTANTE:

Esta arquitetura NÃO deve ser tratada como definitiva.

Ela é apenas uma referência para discussão.

Analise o código e proponha melhorias caso encontre uma estrutura mais adequada.

MODELO HÍBRIDO

Não assumir que todos os módulos setoriais precisam desaparecer.

Especialmente o Financeiro deve provavelmente continuar existindo como uma visão transversal.

Isso ocorre porque o usuário financeiro pode precisar executar tarefas como:

- ver todos os Contas a Receber;
- ver todos os Contas a Pagar;
- fechar uma competência;
- acompanhar caixa;
- realizar conciliação;
- analisar DRE;
- verificar inadimplência;

independentemente da origem do lançamento.

Portanto, avaliar um modelo híbrido:

VISÃO POR FLUXO

- Operações por Volume
- CLT
- Intermitentes
- Diaristas
- Serviços Extras
- Custos Extras

VISÕES TRANSVERSAIS

- Financeiro
- Relatórios
- Cadastros
- Configurações

PERMISSÕES

A nova arquitetura NÃO pode significar que todos os usuários verão todas as opções de cada fluxo.

A navegação deve continuar respeitando as permissões atuais.

Exemplo conceitual:

Encarregado dentro de Operações por Volume

Pode visualizar:

- Visão Geral
- Nova Operação
- Minhas Operações

RH no mesmo módulo

Pode visualizar:

- Visão Geral
- Operações
- Aprovações
- Pendências relacionadas ao RH

Admin

Pode visualizar:

- Visão Geral
- Nova Operação
- Operações
- Aprovações
- Pendências
- Histórico

Financeiro

Pode visualizar:

- Operações aprovadas
- Faturamento
- Contas a Receber relacionadas

Verifique como as permissões estão implementadas atualmente:

- por role;
 - por rota;
 - por componente;
 - por menu;
 - por permission key;
 - por tenant;
 - por combinação desses mecanismos.
-

FASE 1 — MAPEAR A ARQUITETURA ATUAL

Identifique exatamente como a navegação atual está construída.

Mapear:

1. Componente principal da Sidebar.
2. Configuração dos menus.
3. Rotas principais.
4. Rotas aninhadas.
5. Layouts.
6. Guards.
7. Permission wrappers.
8. Breadcrumbs.
9. Deep links.
10. Redirecionamentos.
11. Rotas legadas.
12. Menus condicionais por perfil.
13. Dependências entre rota e regra de negócio.

Gerar uma árvore da navegação atual.

FASE 2 — MAPEAR OS FLUXOS REAIS

Mapear individualmente os seguintes fluxos:

1. Operação por Volume
2. CLT
3. Intermitentes
4. Diaristas
5. Serviços Extras
6. Custos Extras

Para cada um, informar:

- ponto de entrada;
- página inicial;
- usuário responsável;
- status utilizados;
- aprovações;
- telas envolvidas;
- páginas de RH;

- lotes;
- cálculo de horas;
- financeiro;
- faturamento;
- pagamentos;
- histórico;
- finalização.

Representar cada fluxo em formato:

INPUT

→ ETAPA

→ USUÁRIO

→ TELA

→ STATUS

→ PRÓXIMA ETAPA

FASE 3 — IDENTIFICAR COMPONENTES COMPARTILHADOS

Identificar quais funcionalidades são reutilizadas entre diferentes fluxos.

Exemplos a investigar:

- Aprovações
- Lotes
- Jornadas
- Horas
- Financeiro
- Faturamento
- Histórico
- OperacaoForm
- Tabelas
- Filtros
- Modais
- Services
- Hooks

Para cada componente compartilhado informar:

- arquivo;
- componente;
- fluxos que utilizam;
- dependências;

- possibilidade de receber um contexto/filtro;
 - risco de reutilização;
 - necessidade ou não de refatoração.
-

FASE 4 — MAPEAR ACOPLAMENTOS

Identifique onde existe acoplamento entre:

ROTA

→ COMPONENTE

→ REGRA DE NEGÓCIO

Precisamos descobrir se mudar a localização visual de uma página pode afetar seu funcionamento.

Investigue especialmente:

- uso de `pathname`;
 - verificações baseadas em URL;
 - breadcrumbs;
 - autorização baseada em rota;
 - redirect pós-save;
 - redirect pós-aprovação;
 - links internos;
 - links enviados em notificações;
 - E2E;
 - query parameters;
 - estado salvo na URL;
 - filtros dependentes de rota.
-

FASE 5 — ROTAS E ALIASES

Avaliar se podemos preservar as rotas atuais e criar novas portas de entrada.

Exemplo:

Rota atual:

`/rh/aprovacoes`

Nova rota contextual:

/diaristas/aprovacoes

Ambas eventualmente podem utilizar o mesmo componente.

Investigar estratégias como:

- route alias;
- wrapper de página;
- query parameter;
- prop de contexto;
- nested routes;
- redirect controlado;
- reutilização do mesmo component;
- menu contextual.

Indicar a alternativa mais segura.

FASE 6 — MAPA ATUAL → PROPOSTO

Gerar tabela contendo:

Fluxo	Tela atual	Rota atual	Nova categoria proposta	Nova rota sugerida	Reutiliza componente?	Risco
-------	------------	------------	-------------------------	--------------------	-----------------------	-------

Fazer isso para todas as telas relevantes.

FASE 7 — IDENTIFICAR O QUE NÃO DEVE MUDAR

Criar seção específica chamada:

CONTRATO DE NÃO REGRESSÃO

Listar tudo que deve permanecer intacto.

No mínimo investigar:

- tabelas;
- schemas;
- RLS;

- tenants;
 - status;
 - motores financeiros;
 - cálculos;
 - RPCs;
 - Edge Functions;
 - integrações;
 - workflows externos;
 - importações;
 - aprovações;
 - geração de lotes;
 - jornada;
 - pagamento;
 - faturamento;
 - CNAB;
 - DRE;
 - logs;
 - auditoria.
-

FASE 8 — ANALISAR RISCOS

Classificar riscos em:

BAIXO

Exemplo:

- mudança de label;
- reorganização da sidebar;
- agrupamento visual.

MÉDIO

Exemplo:

- aliases de rotas;
- wrappers;
- breadcrumbs;
- filtros contextuais.

ALTO

Exemplo:

- permissões dependentes de URL;

- regras de negócio dentro de componentes de navegação;
- redirects hardcoded;
- testes E2E dependentes das rotas atuais;
- integrações que utilizam URLs internas.

Para cada risco informar:

- causa;
 - impacto;
 - como mitigar.
-

FASE 9 — PROPOSTA DE NOVA ARQUITETURA DA INFORMAÇÃO

Após entender o código atual, propor a melhor arquitetura possível.

Não simplesmente reproduzir a estrutura sugerida acima.

Avaliar:

- facilidade de aprendizado;
- número de cliques;
- consistência;
- frequência das tarefas;
- clareza para novos usuários;
- distinção entre fluxo e setor;
- necessidade de dashboards;
- excesso de submenus;
- permissões;
- escalabilidade futura.

Gerar uma árvore final recomendada.

FASE 10 — PIPELINE VISUAL POR MÓDULO

Avaliar a possibilidade de cada módulo ter uma “Visão Geral” contextual mostrando o estado do fluxo.

Exemplo Diaristas:

Lançados

- Aguardando Aprovação
- Aprovados
- Em Lote
- Aguardando Pagamento
- Pagos

Exemplo Intermitentes:

Importados

- Jornadas Processadas
- Inconsistências
- Aprovação
- Lote
- Pagamento

Não implementar.

Apenas avaliar:

- viabilidade;
- dados disponíveis;
- queries existentes;
- necessidade de novas queries;
- custo técnico;
- benefício de UX.

FASE 11 — ESTRATÉGIA DE MIGRAÇÃO

Propor uma implementação incremental que permita testar sem desmontar a navegação atual.

Preferencialmente analisar algo semelhante a:

Etapas A

Criar a nova árvore de navegação sem remover a antiga.

Etapas B

Criar aliases/contextos para páginas existentes.

Etapas C

Aplicar filtros específicos por fluxo.

Etapa D

Validar permissões.

Etapa E

Executar testes E2E e homologação manual.

Etapa F

Remover ou ocultar menus antigos somente após homologação.

Essa sequência é apenas uma referência.

Proponha a estratégia tecnicamente mais segura com base no projeto real.

FASE 12 — TESTES DE NÃO REGRESSÃO

Criar posteriormente um plano de testes para garantir que a reorganização não alterou os fluxos.

Deve incluir:

- Operação por Volume
- CLT
- Intermitente
- Diarista
- Serviço Extra
- Custo Extra
- RH
- Admin
- Encarregado
- Financeiro
- Tenant
- Permissões
- Faturamento
- Pagamentos
- DRE
- Lotes
- Jornada

Mas nesta etapa apenas elaborar o plano.

NÃO executar alterações ainda.

FORMATO OBRIGATÓRIO DA RESPOSTA

Entregar um documento contendo exatamente esta estrutura:

1. RESUMO EXECUTIVO

Explique se a mudança é:

- viável;
- parcialmente viável;
- arriscada;

e por quê.

2. COMO O ORBE ESTÁ ORGANIZADO HOJE

Árvore real baseada no código.

3. COMO OS FLUXOS FUNCIONAM HOJE

Mapa real dos seis fluxos.

4. PROBLEMAS DE UX IDENTIFICADOS

Baseados na arquitetura atual.

5. COMPONENTES E FUNCIONALIDADES COMPARTILHADAS

Lista técnica com arquivos.

6. MAPA DE ROTAS

Rota atual → componente → permissões.

7. ACOPLAMENTOS E RISCOS

Análise detalhada.

8. PROPOSTA DE NOVA ARQUITETURA

Árvore recomendada.

9. MAPA DE MIGRAÇÃO

Atual → Novo.

10. PERMISSÕES NA NOVA ORGANIZAÇÃO

Por perfil.

11. O QUE NÃO PRECISA SER ALTERADO

Backend, banco, regras etc.

12. O QUE PRECISA SER ADAPTADO

Frontend/navegação.

13. ESTRATÉGIA DE IMPLEMENTAÇÃO

Fases incrementais.

14. TESTES DE NÃO REGRESSÃO

Plano.

15. DÚVIDAS E DECISÕES QUE PRECISAM SER DISCUTIDAS

Liste explicitamente todos os pontos em que existe mais de uma solução possível.

RESTRIÇÃO FINAL

NÃO implemente nenhuma mudança após a auditoria.

NÃO considere a arquitetura proposta como aprovada.

NÃO faça refatoração automática.

NÃO altere nenhum fluxo funcional.

NÃO elimine rotas.

NÃO altere banco.

NÃO altere permissões.

NÃO faça migrations.

NÃO altere Edge Functions.

NÃO altere motores financeiros.

NÃO altere integrações.

NÃO altere testes apenas para fazê-los passar.

Ao finalizar, pare e apresente o relatório.

A próxima etapa será uma discussão técnica entre nós para refinarmos a arquitetura antes de autorizar qualquer implementação.

PRINCÍPIO DE UX QUE DEVE ORIENTAR TODA A ANÁLISE

O usuário não deve precisar compreender a arquitetura interna do ORBE para conseguir concluir um processo.

O sistema deve organizar a experiência prioritariamente em torno do objeto que o usuário está tratando:

- CLT
- Intermitente
- Diarista
- Operação por Volume
- Serviço Extra
- Custo Extra

Enquanto setores, permissões, motores financeiros, regras, infraestrutura e responsabilidades continuam existindo internamente.

Quando o Antigravity devolver a auditoria, o ideal é **não autorizarmos a implementação imediatamente**. Primeiro podemos comparar a árvore real encontrada por ele com essa

proposta, decidir módulo por módulo o que deve ficar condensado e só então gerar o prompt de implementação incremental.